Google Summer of Code, 2026

Proposal for Checkstyle

Project: Improve Google Style coverage on false negatives





Name: Atharv Chavan

GitHub: Atharv3221

LinkedIn: Atharv Chavan

Email: atharv55215@gmail.com

Timezone: Indian Standard Time (IST), UTC +5:30

Project Size: Large

Mentors: Mauryan Kansara, Mohit, Roman Ivanov

Table Of Contents

About Me……3

Project Details……4

How is the Google Style Guide Covered.....5

False negatives in google style guide coverage……..6

Test Support for google guide coverage……...8

Project Timeline……...9

Expectations from Mentors……11

Contributions so far……12

Why Me?.....14

Future Work……..15

About Me

I'm Atharv Chavan, a third-year B.Tech CSE student aiming to become a software developer. I got into open source because I wanted to understand how large systems actually work, and Checkstyle has been my ground for that over the past year.

In that time I've contributed across different areas implementing LineEndingCheck, redesigning an existing check, fixing bugs, resolving mutation coverage gaps, handling Error Prone warnings, and working on CI improvements. I've also made a few early contributions to Apache Camel.

This work taught me a lot, not just about Java or testing, but also how decisions are made in a codebase and how a real open source community works.

Project Details

During GSoC'25, google style guide was successfully implemented to the latest version of Apr 25, 2025. google_checks.xml is the configuration file where google java style guide is implemented. Though almost all rules of the google style guide are covered. Users have reported bunch of issues pointing out flaws in the implementation, these issues are labeled as google style issues, there is need of solving them, the majority of such issues are "false-negatives". Also Google Style Guide is updated in September 2025, there is a need of updating coverage page to match the latest requirements.

Project Deliverables:

- Resolve all issues labeled as google style.
- Reduce "Blue Tick" rules by analyzing the rule and our coverage for that rule and provide solutions for them.
- Investigate the issues related to Modules/Checks in issue tracker of Checkstyle repository which are used in google_checks.xml, report to mark them with google style label and solve them.

How is the Google Style Guide Covered?

XML files are used to configure Checkstyle checks based on requirements. Google style guide is implemented using configuration file google_checks.xml. This file contains different modules/checks and suppressions that are used to enforce the rules of the style guide. The Configuration page provides details on how Checkstyle checks can be configured.

The style guide rules are implemented using different Checkstyle checks, each handling a specific part of the rules. The coverage table on the coverage_page shows how each rule is implemented using one or more checks, and also shows if each rule is covered, partially covered or can't cover. Sometimes a single rule in the style guide is not directly covered by a single Checkstyle check. In such cases, multiple checks are used to enforce the rule effectively, and suppression are used to fine-tune or limit the behavior of these checks so that the implementation aligns more accurately with the google style guideline.

Overall, a combination of different checks and suppressions is used to enforce the google style guide.

False negatives in google style guide coverage

Rule	Checks	Reason	Issue
3 Source file structure	EmptyLineSeparator	Lack of support for Java modules Grammar	#8240
3.5.1 Ordering and spacing of module directives	--	Lack of support for Java modules Grammar	#17170
4.1.2 Nonempty blocks: K & R style	LeftCurly RightCurly	There are some false-negatives regarding ELSE and DEFAULT blocks	#15791 #14782
4.1.3 Empty blocks: may be concise	RegexpSinglelineJava	There are some false-negatives regarding K & R style brackets for empty blocks,	#15791
4.4 Where to break	SeparatorWrap	Line break before and after the lambda(->) is not covered.	#17253
4.6.2 Horizontal whitespace	WhiteSpaceAround	A single whitespace before the empty block({}) of a class, method, interface, static, loops(while, do, for), and record is not covered.	#17715
4.8.5.2 Class, package, and module annotations	InvalidJavadocPosition	Lack of support for Java modules Grammar	#8240

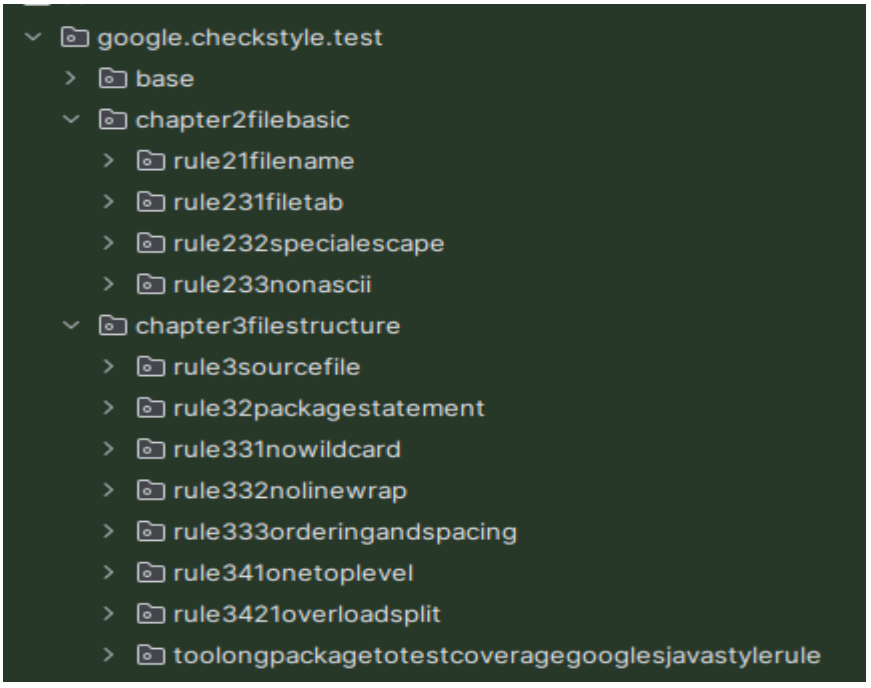
5.2.1 Package and modules names	PackageName	Lack of support for Java modules Grammar	#8240
5.2.3 Method names	MethodName	There are some false-negatives regarding the use of underscores in the method names	#17841
5.3 Camel case: defined	AbbreviationAsWordInName	There are some false-negatives regarding the use of underscores in method and member names	#17841 #17842
7.1.1 General form	JavadocContentLocation InvalidJavadocPosition	There is a false negative related to Javadoc content that begins with a leading asterisk There is a false negative concerning Javadoc blocks where the closing */ does not appear on its own line	#18273

Test Support for google guide coverage

Test classes extends abstract class AbstractGoogleModule -TestSupport and test class for indentation extends AbstractIndentationTestSupport.

“verifyWithWholeConfig()” method is used for checking violations of a resource file with config google_checks.xml are recorded and matched with the violation comments in the file. A formatted file is expected with matching name InputFormatted*.java where it's expected to have no violations and all valid changes according to google-java-formater are covered.

The package structure is organized by chapter and rule number as defined in the Google Java Style Guide, where each chapter package groups related rules and each rule package corresponds to a specific rule number and its keyword.



Project Timeline

Weeks	Start Date	End Date	Tasks to be complemented
-	May 1	May 24	Community Bounding Period
1	May 25	May 28	Go through all reported issues, and report them as google style if their module is used in google_checks.xml
2	May 29	June 5	Add support for java modules Grammar
3	June 6	June 25	Resolve issue #17170 and update checks which are stuck due to #8240
4	June 26	June 29	Resolve issues for 4.1.2 Nonempty blocks: K & R style #15791 #14782
5	June 30	July 4	Resolve 4.4 Where to break #17253
6	July 5	July 14	Resolve 4.6.2 Horizontal whitespace #17715,
7	July 15	July 18	Resolve 5.2.3 Method names #17841
8	July 18	July 24	Resolve 5.3 Camel case: defined #17842
9	July 25	August 1	Resolve 7.1.1 General form #18273
10	August 1	August 14	Cover all false negatives issues reported for the modules used in google_checks.xml

11	August 15	August 21	Cover all false positive issues reported for the modules used in google_checks.xml
12	August 22	August 24	Project review and final reports submitted to mentors

Note: Coverage documentation is updated continuously as rule related issues are resolved.

I have assigned different numbers of days to each section based on their requirements. I will be committing 4 - 5 hours daily to complete the project before the deadline. My exams run from April 21 to May 5, which overlaps with the start of the community bonding period, but since coding begins May 25 I will be fully available for the entire coding period. During the community bonding period, I will discuss any unclear requirements with maintainers and start sending PRs as early as possible.

Expectations from Mentors

This project involves implementing changes across modules and their functionalities. Maintainer guidance is expected primarily on implementation decisions, including design choices and the rationale behind preferring one approach over alternatives. Maintainers are also expected to review new functionalities or modules and provide timely feedback to ensure tasks are completed efficiently.

Contributions so far

Modules and functionalities:

PR	Title	Status
#18393	Adding new check LineEnding	Merged
#18702	Adding new check UnnecessaryPermitsClause	Working
#18567	Rework OneStatementPerLine Implementation Logic	Reviewing
#19550	Adding new Check RedundantThis	Working
#19073	False negatives fixed for pattern variables	Reviewing
#17489	Fixed False positive for JavadocType:Unused @param tag	Merged

CI Issues:

PR	Title	Status
#17280	Using qodana for inspection	Merged
#17374	Updated google-java-format.sh to validate difference in size is ~50 bytes	Merged
#17418	Added Trino to no error testing in CI	Merged
#17325	Removed from Travis CI snapshot deploy	Merged

Pitest Issues:

PR	Title	Status
#16592	Resolved Pitest Suppression in Pitest-Javadoc Profile of JavadocMethodCheck	Merged
#16699	Cover pitest survivals with tests in DetailNodeTreeStringPrinter	Merged
#16718	Cover pitest survivals in SuppressionStringPrinter	Merged
#16840	Covered pitest survivals for SuppressFilterElement	Merged
#16852	Covered pitest survivals with tests for CheckstyleAntTask	Merged
#16873	Covered pitest survivals in VariableDeclarationUsageDistanceCheck	Merged

Other:

PR	Title	Status
#17857	Resolved part of error-prone violations	Merged
#16764	Resolved CheckerFramework violation	Merged
#16956	Used nio api in CheckUtil	Merged
#1016	Activate test in jdk21	Merged
#232	Added apache/pulsae and RedHatInsights/rhsm-subscriptions to project list	Merged

Here are all my Prs

Issues I opened: link

Why Me?

I have been actively contributing to Checkstyle since February 2025 and have spent the last 2–3 weeks researching this project with a clear plan to complete it.

I have worked across various parts of the codebase, implementing new modules, reworking existing ones, fixing bugs, adding new features, improving CI, resolving Error Prone warnings, and working on different types of check implementations. This breadth of experience gives me a strong understanding of how Checkstyle works internally and makes me well-equipped to handle the diverse tasks this project demands.

This project requires deep knowledge of how different modules work, how existing checks can be extended, and how style guide coverage is implemented and tested areas I have directly worked on. With this experience, I am confident that I can efficiently handle this project and deliver it on time.

Future Work

Contributing to Checkstyle has strengthened my skills in testing, debugging, writing clean code, and CI/CD practices. It has also improved my problem-solving ability and helped me gain experience working with a large, real-world codebase.

Going forward, I aim to deepen my knowledge by contributing to more complex modules, improving code quality, and working on grammar and to enhance overall project quality. In the long term, I would like to broaden my contributions to related projects while continuing to actively contribute to Checkstyle.

THANK YOU